Object Oriented Design Quality Metrics

References

- Analyze java package metrics in a graph database
- Calculate metrics
- jqassistant
- notebook walks through examples for integrating various packages with Neo4j
- OO Design Quality Metrics
- A Validation of Martin's Metric
- Neo4j Python Driver

Incoming Dependencies

Incoming dependencies are also denoted as "Fan-in", "Afferent Coupling" or "in-degree". These are the ones that use the listed package.

If these packages get changed, the incoming dependencies might be affected by the change. The more incoming dependencies, the harder it gets to change the code without the need to adapt the dependent code (“rigid code”). Even worse, it might affect the behavior of the dependent code in an unwanted way (“fragile code”).

Since Java Packages are organized hierarchically, incoming dependencies can be count for every package in isolation or by including all of its sub-packages. The latter one is done without top level packages like for example "org" or "org.company" by assuring that only packages are considered that have other packages or types in the same hierarchy level ("siblings").

Table 1a

- Show the top 20 Typescript modules with the most incoming dependencies
- Set the property "incomingDependencies" on Module nodes if not already done.

	fullQualifiedModuleName	moduleName	incomingDependencies	incomingDependenciesWeight	incomingDependentAbstractTypes	incomingD
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	7	280	0	
1	/home/runner/work/code-graph-analysis-examples...	react-router-dom-v5-compat	0	0	0	
2	/home/runner/work/code-graph-analysis-examples...	server	0	0	0	
3	/home/runner/work/code-graph-analysis-examples...	react-router-native	0	0	0	
4	/home/runner/work/code-graph-analysis-examples...	react-router	0	0	0	
5	/home/runner/work/code-graph-analysis-examples...	router	0	0	0	

Outgoing Dependencies

Outgoing dependencies are also denoted as "Fan-out", "Efferent Coupling" or "out-degree". These are the ones that are used by the listed package.

Code from other packages and libraries you’re depending on (outgoing) might change over time. The more outgoing changes, the more likely and frequently code changes are needed. This involves time and effort which can be reduced by automation of tests and version updates. Automated tests are crucial to reveal updates, that change the behavior of the code unexpectedly (“fragile code”). As soon as more effort is required, keeping up becomes difficult (“rigid code”). Not being able to use a newer version might not only restrict features, it can get problematic if there are security issues. This might force you to take “fast but ugly” solutions into account which further increases technical dept.

Since Java Packages are organized hierarchically, outgoing dependencies can be count for every package in isolation or by including all of its sub-packages. The latter one is done without top level packages like for example "org" or "org.company" by assuring that only packages are considered that have other packages or types in the same hierarchy level ("siblings").

Table 2a

- Show the top 20 Typescript modules with the most outgoing dependencies
- Set the "outgoingDependencies" properties on Module nodes if not already done

	fullQualifiedModuleName	sourceName	outgoingDependencies	outgoingDependenciesWeight	outgoingDependentAbstractTypes	outgoingDe
0	/home/runner/work/code-graph-analysis-examples...	react-router-dom	156	10765	0	
1	/home/runner/work/code-graph-analysis-examples...	server	39	1650	0	
2	/home/runner/work/code-graph-analysis-examples...	react-router-native	24	764	0	
3	/home/runner/work/code-graph-analysis-examples...	react-router	17	261	0	
4	/home/runner/work/code-graph-analysis-examples...	react-router-dom-v5-compat	0	0	0	
5	/home/runner/work/code-graph-analysis-examples...	router	0	0	0	

Instability

$$Instability = \frac{Outgoing\ Dependencies}{Outgoing\ Dependencies + Incoming\ Dependencies}$$

Instability is expressed as the ratio of the number of outgoing dependencies of a module (i.e., the number of packages that depend on it) to the total number of dependencies (i.e., the sum of incoming and outgoing dependencies).

Small values near zero indicate low *Instability*. With no outgoing but some incoming dependencies the Instability is zero which is denoted as maximally stable. Such code units are more rigid and difficult to change without impacting other parts of the system. If they are changed less because of that, they are considered stable.

Conversely, high values approaching one indicate high *Instability*. With some outgoing dependencies but no incoming ones the *Instability* is denoted as maximally unstable. Such code units are easier to change without affecting other modules, making them more flexible and less prone to cascading changes throughout the system. If they are changed more often because of that, they are considered unstable.

Since Java Packages are organized hierarchically, *Instability* can be calculated for every package in isolation or by including all of its sub-packages.

Table 3a

- Show the top 20 Typescript modules with the lowest *Instability*
- Set the property "instability" on Module nodes if not already done

	projectName	fullQualifiedModuleName	moduleName	instability	instabilityAbstractTypes	instabilityModules	instabilityPackages	module.o
0	react-router-dom-v5-compat	/home/runner/work/code-graph-analysis-examples...	react-router-dom-v5-compat	0.000000	0.0	0.0	0.0	
1	router	/home/runner/work/code-graph-analysis-examples...	router	0.000000	0.0	0.0	0.0	
2	react-router-dom	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0.957055	0.0	0.8	0.0	
3	react-router-dom	/home/runner/work/code-graph-analysis-examples...	server	1.000000	0.0	1.0	0.0	
4	react-router-native	/home/runner/work/code-graph-analysis-examples...	react-router-native	1.000000	0.0	1.0	0.0	
5	react-router	/home/runner/work/code-graph-analysis-examples...	react-router	1.000000	0.0	1.0	0.0	

Abstractness

$$Abstractness = \frac{abstract\ classes\ in\ category}{total\ number\ of\ classes\ in\ category}$$

Package *Abstractness* is expressed as the ratio of the number of abstract classes and interfaces to the total number of classes of a package.

Zero *Abstractness* means that there are no abstract types or interfaces in the package. On the other hand, a value of one means that there are only abstract types.

Since Java Packages are organized hierarchically, *Abstractness* can be calculated for every package in isolation or by including all of its sub-packages.

Table 4a

- Show the top 30 Typescript modules with the lowest *Abstractness*
- Set the property "abstractness" on Module nodes if not already done.

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotifica
tion.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provid
ed property key is not in the database} {description: One of the property names in yo
ur query is not available in the database, make sure you didn't misspell it or that t
he label is available when you run this statement in your application (the missing pr
operty name is: numberAbstractTypes)} {position: line: 10, column: 16, offset: 500} f
or query: "// Get Typscript Modules with the lowest abstractness first (if set befor
e)\n \n MATCH (module:TS:Module)\n WHERE module.abstractness IS NOT NULL \n OPTIONAL
MATCH (projectdir:Directory)<-[:HAS_ROOT]-(project:TS:Project)-[:CONTAINS]->(module)
\n RETURN reverse(split(reverse(projectdir.absoluteFileName), '/')[0]) AS projectName
\n       ,module.globalFqn           AS fullQualifiedModuleName\n       ,module.name
AS moduleName\n       ,module.abstractness        AS abstractness\n       ,module.num
berAbstractTypes AS numberAbstractTypes\n       ,module.numberTypes         AS number
Types\n ORDER BY abstractness ASC, numberTypes DESC\nLIMIT 20"
```

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.UnknownPropertyKeyWarning} {category: UNRECOGNIZED} {title: The provided property key is not in the database} {description: One of the property names in your query is not available in the database, make sure you didn't misspell it or that the label is available when you run this statement in your application (the missing property name is: numberTypes)} {position: line: 11, column: 16, offset: 558} for query: "// Get Typscript Modules with the lowest abstractness first (if set before)\n \n MATCH (module:TS:Module)\n WHERE module.abstractness IS NOT NULL \n OPTIONAL MATCH (projectdir:Directory)<-[:HAS_ROOT]-(project:TS:Project)-[:CONTAINS]->(module)\n RETURN reverse(split(reverse(projectdir.absoluteFileName), '/')[0]) AS projectName\n      ,module.globalFqn           AS fullQualifiedModuleName\n      ,module.name                AS moduleName\n      ,module.abstractness        AS abstractness\n      ,module.numberAbstractTypes AS numberAbstractTypes\n      ,module.numberTypes         AS numberTypes\n ORDER BY abstractness ASC, numberTypes DESC\nLIMIT 20"
```

	projectName	fullQualifiedModuleName	moduleName	abstractness	numberAbstractTypes	numberTypes
0	router	/home/runner/work/code-graph-analysis-examples...	router	0.000000	None	None
1	react-router-dom-v5-compat	/home/runner/work/code-graph-analysis-examples...	react-router-dom-v5-compat	0.000000	None	None
2	react-router-dom	/home/runner/work/code-graph-analysis-examples...	react-router-dom	0.206349	None	None
3	react-router-native	/home/runner/work/code-graph-analysis-examples...	react-router-native	0.277778	None	None
4	react-router-dom	/home/runner/work/code-graph-analysis-examples...	server	0.333333	None	None
5	react-router	/home/runner/work/code-graph-analysis-examples...	react-router	0.714286	None	None

Distance from the main sequence

The *main sequence* is a imaginary line that represents a good compromise between *Abstractness* and *Instability*. A high distance to this line may indicate problems. For example is very *stable* (rigid) code with low abstractness hard to change.

Read more details on that in OO Design Quality Metrics and Calculate metrics.

Table 5a

- Show the top 30 Typescript modules with the highest distance from the "main sequence"

	artifactName	fullQualifiedName	name	distance	abstractness	instability	elementsCount
0	router	/home/runner/work/code-graph-analysis-examples...	./index.ts	1.000000	0.000000	0.000000	0
1	react-router-dom-v5-compat	/home/runner/work/code-graph-analysis-examples...	./index.ts	1.000000	0.000000	0.000000	0
2	react-router	/home/runner/work/code-graph-analysis-examples...	./index.ts	0.714286	0.714286	1.000000	7
3	react-router-dom	/home/runner/work/code-graph-analysis-examples...	./server.tsx	0.333333	0.333333	1.000000	6
4	react-router-native	/home/runner/work/code-graph-analysis-examples...	./index.tsx	0.277778	0.277778	1.000000	18
5	react-router-dom	/home/runner/work/code-graph-analysis-examples...	./index.tsx	0.163404	0.206349	0.957055	63

Abstractness vs. *Instability* Plot with "Main Sequence" line as reference

- Plot *Abstractness* vs. *Instability* of all packages
- Draw the "main sequence" as dashed green diagonal line
- Scale the packages by the number of types they contain
- Color the packages by their distance to the "main sequence" (blue=near, red=far)

Figure 5a - Typescript Modules